Carnegie Mellon University
Master of
Software Engineering

LG Software Architectures Training Program - Project Description

Time Grapher

Introduction and Background

A timegrapher is a diagnostic instrument used to evaluate the performance of a mechanical watch. It listens to the ticking of the movement through a microphone and analyzes the resulting timing signals to estimate how well the watch is running. If you have ever seen a watchmaker place a watch on a small machine that displays scrolling lines and numerical readings, that machine is a timegrapher. A timegrapher typically reports key measurements such as rate, amplitude, beat error, and beat rate, and it also produces visual traces that help users judge stability, consistency, and possible mechanical faults.

Several of these measurements are especially important. Rate is shown in seconds per day and indicates how fast or slow the watch is running compared with ideal time. For example, +5 s/d means the watch is gaining five seconds per day, while -12 s/d means it is losing twelve seconds per day. Amplitude measures the angular swing of the balance wheel in degrees and is one of the best indicators of movement health and available power. As a rough guideline, 270° to 310° is often considered strong for many modern watches, 220° to 250° may be acceptable but can suggest wear or need for service, and values below 200° often indicate a problem. Beat error measures how symmetrical the tick and tock are; 0.0 ms is ideal, values under 0.6 ms are generally considered good, and higher values may suggest the need for adjustment or service. A timegrapher also displays the watch's beat rate, or beats per hour (BPH), which reflects the vibration frequency of the movement. Common examples are 21,600 bph, 28,800 bph, and 36,000 bph.

In addition to these numerical values, the machine creates visual traces. Straight, clean lines usually indicate stable performance, while jagged, wandering, or inconsistent traces can suggest positional variation, low amplitude, escapement problems, magnetism, or mechanical wear. Even so, a timegrapher does not tell the whole story of real-world timekeeping. A watch may perform well on a timegrapher and still behave differently on the wrist because of position changes, power-reserve effects, shocks, or temperature. Nevertheless, it remains one of the fastest and most useful tools for evaluating watch condition, diagnosing problems, and guiding regulation or service. Collectors,

enthusiasts, and watchmakers use time graphers to check watch health, compare positional performance, diagnose problems before service, regulate movements, and evaluate the condition of a watch before purchase.

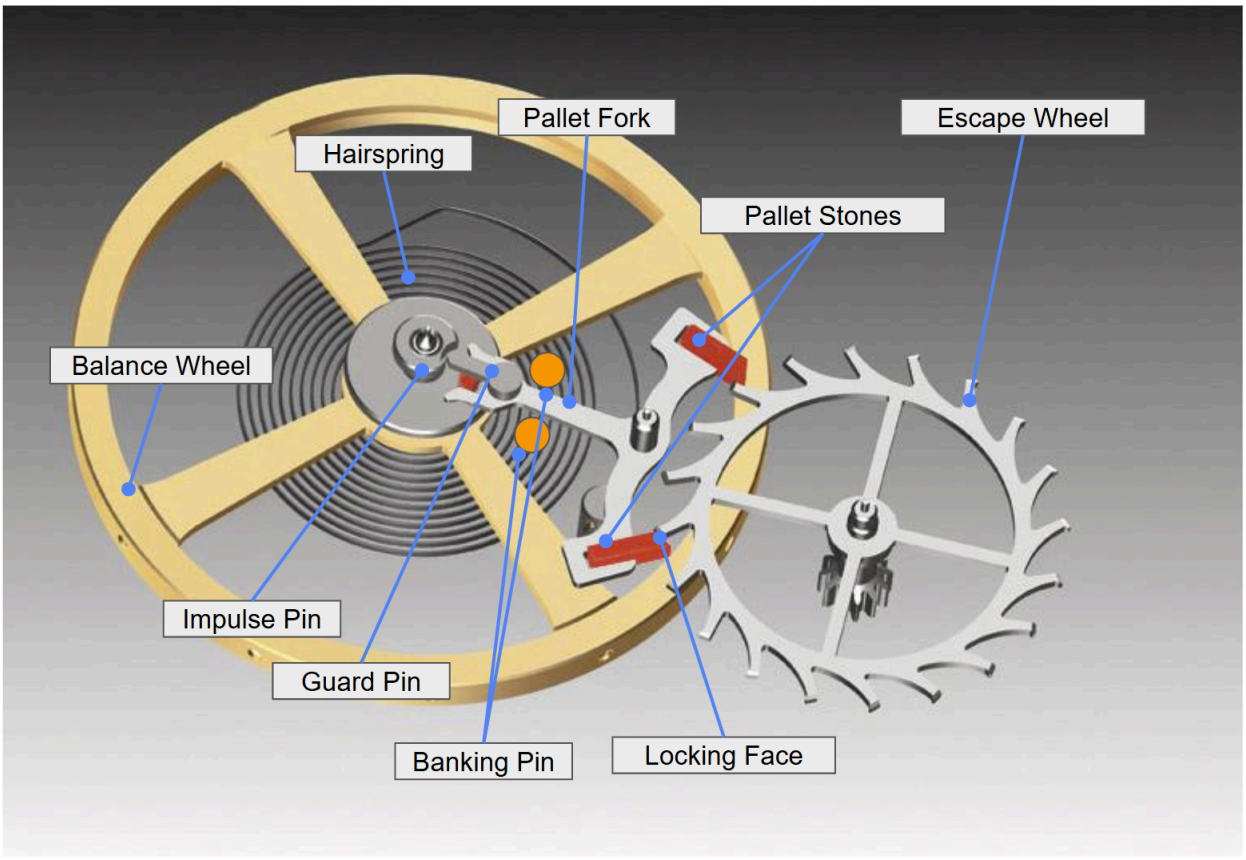

Figure 1[1]: Swiss Lever Escapement

For a Swiss lever escapement, the watch produces three distinct acoustic events during each beat. In this document, these signals are labeled T1, T2, and T3, and may also be referred to as A, B, and C.

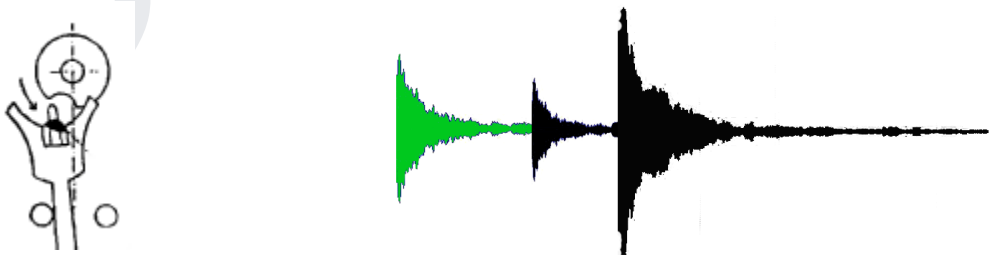

Figure 2: Impulse Pin Strikes Fork, First Noise in Sequence

[1] The following provides a good overview:https://youtu.be/JgdO8kzuwig?si=0bNXTXQ3qEVSx_aC

T1 (A) is the first noise and occurs when the impulse pin strikes the pallet fork. This event is generally the most precise and repeatable in time, making it especially useful for measurement; because of its timing consistency, it is used to determine rate and beat error.

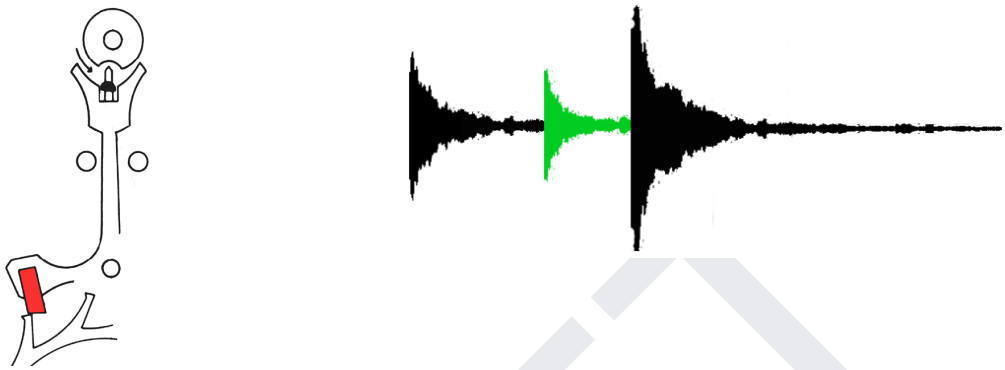

Figure 3: Escape Wheel Tooth Slides, Second Noise in Sequence

T2 (B) is the second noise and occurs when an escape wheel tooth slides on the pallet stone during impulse transfer. This signal is relatively irregular and inconsistent and is therefore not typically used for measurement.

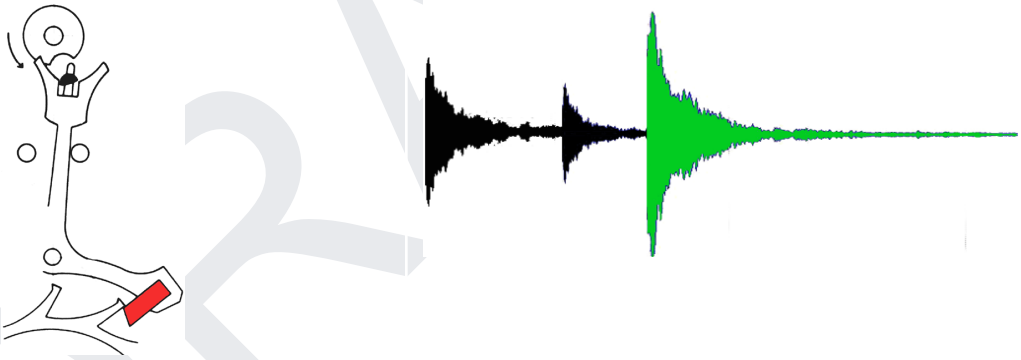

Figure 4: Escape Wheel Locks on Pallet, Third Noise in Sequence

T3 (C) is the third noise and occurs when the escape wheel tooth locks on the pallet and the pallet fork strikes the banking pin. This is usually the strongest sound and is used, together with T1, to calculate amplitude.

By detecting these key acoustic events, the system can estimate several important watch-performance measures. Using the measured timing of the watch beats, the software can compute rate (overall timekeeping accuracy), rate deviation (how much the watch gains or loses relative to the nominal rate), and beat error (how symmetric the two half-oscillations are). Using the measured interval between the A and C events of the same beat, together with the watch's beat rate and configured lift angle, the system can also estimate amplitude, which reflects the angular swing of the balance wheel. In addition, the system uses the watch's beats per hour to determine the balance-wheel frequency. These measures form the basis of the graphs and numerical displays

described in this project. Students should refer to the TimeGrapher Equations document for the detailed formulas, assumptions, and worked examples.

Project Overview

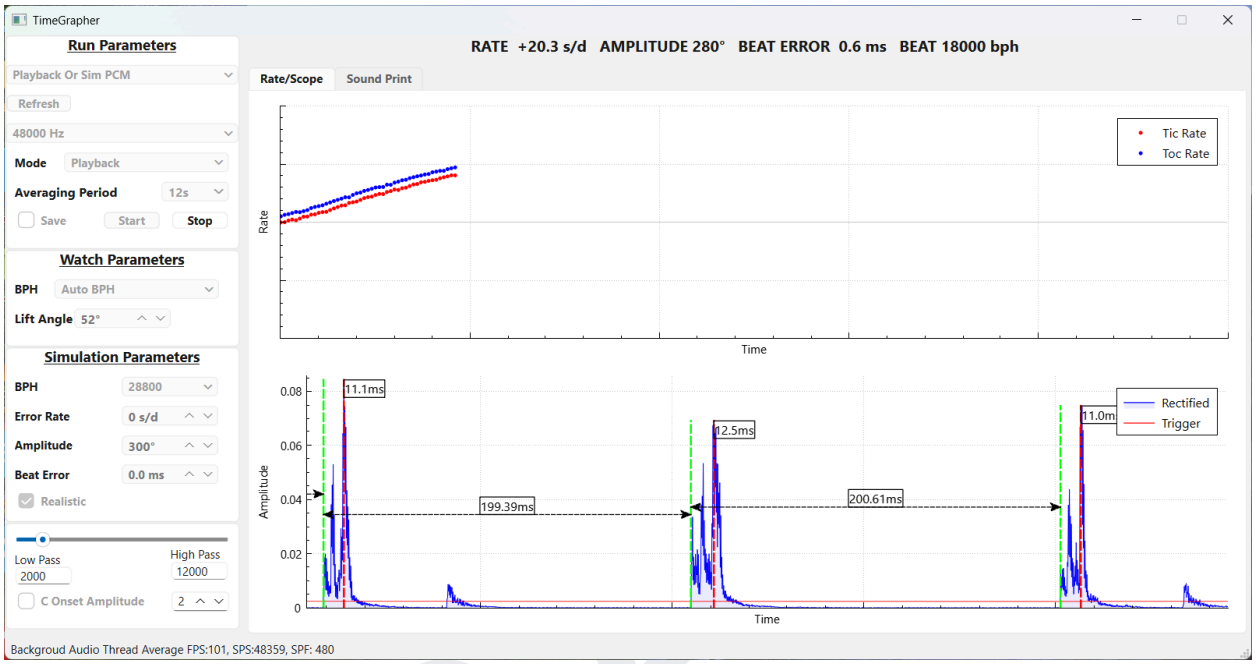

Figure 5: Example Graphical User Interface for the TimeGrapher Project

Objective

This project focuses on capturing acoustic signals from a mechanical watch, extracting meaningful timing and mechanical performance data from those signals, and processing and visualizing the results in real time. In short, the system performs:

- Signal detection
- Signal processing
- Real-time analysis
- Real-time graphing

This project aims to transform a baseline mechanical-watch timegrapher into a real-time diagnostic and visualization system that runs on a Raspberry Pi. Starting from provided code that performs measurements, student teams will design and implement software that detects key beat events, filters and processes acoustic data in real time, and computes meaningful watch-performance measures such as rate, seconds-per-day deviation, beat error, amplitude, lift angle, beats per hour, and balance frequency. The system will present both raw and processed data through an interactive graphical user interface (GUI) with signal markers and real-time graphs that help users interpret watch behavior and verify that the measurements are correct.

As a software architecture project, the goal is not only to make these measurements work, but to create a modular, extensible, and maintainable architecture that supports substantial growth beyond the baseline system. Student teams will add new analyses, GUI capabilities, and real-time visualizations described later in this document, including position-based testing, rate- and amplitude-stability graphs, beat and sequence displays, alerts, and time-series summaries. Because the project is time-boxed to approximately five weeks, teams are expected to prioritize and deliver a feasible, well-architected subset of these capabilities while demonstrating accurate measurement, low latency, and real-time performance on the target hardware.

System Description

Usability and User Purpose

The purpose of the timegrapher GUI is to help a user observe, interpret, and diagnose the performance of a mechanical watch through its acoustic signal. From the user's point of view, the system is not just a graphing tool; it is a diagnostic aid that helps determine whether a watch is running accurately, whether its beat pattern is stable, whether its amplitude is healthy, and whether there are signs of mechanical problems that may require adjustment or repair. The GUI should therefore present raw and processed data in a form that is easy to understand and useful for decision making.

The GUI should support ease of use by clearly showing the current signal, the detected measurement points, and the calculated values that matter most to the user, such as rate, beat error, amplitude, and related summaries. Because users rely on the display to judge the watch's condition, the interface should make it easy to understand what is happening in real time and how to interpret the results. Graphs, labels, markers, alerts, and explanatory text should work together so that a user can quickly identify whether the watch is healthy, unstable, noisy, or out of tolerance (daily timekeeping deviation is falling outside an acceptable range).

The system should also help the user detect faults and interruptions in measurement. If there is a break in continuity, such as loss of signal, missed beats, excessive ambient noise, or an out-of-range reading, the GUI should communicate that condition clearly rather than leaving the user to guess whether the watch or the software is at fault. In these situations, the system should provide meaningful status or error feedback, preserve the last useful reading when appropriate, and guide the user toward recovery, such as repositioning the watch, reducing noise, restarting measurement, or adjusting settings.

Because the watch is measured acoustically, usability also depends on filtering and signal clarity. The GUI and supporting software should help the user obtain a readable measurement by reducing extraneous noise, such as nearby speech, while preserving the watch features needed for correct detection and analysis. In this way, the tool becomes more practical in real usage conditions and more forgiving of imperfect environments.

Overall, the GUI should be designed so that a user can answer practical questions quickly: Is the watch running fast or slow? Is the signal stable? Is the amplitude in a healthy range? Is there a beat-error problem? Has the measurement been interrupted or corrupted by noise? By making these answers visible and understandable, the system supports both effective watch diagnosis and a better user experience.

Graphical User Interface

Figure 5 shows the baseline graphical user interface (GUI) that will be provided to student teams. The baseline GUI displays the raw acoustic signal from the mechanical watch in real time on a Windows PC or Raspberry Pi.

Current Features

Measurement Summary Bar

In the top of of the window, the GUI computes and displays core watch measurements in real time, including rate in seconds-per-day deviation (s/d), amplitude in degrees (deg), beat error in milliseconds (ms), and beats per hour (bph).

Beneath the Measurement Summary Bar is the Tabbed Graph Panel, which currently contains two implemented graph tabs. This panel is the primary area of the GUI for presenting visual analysis and is an ideal location for student teams to add new tabs for additional graphs and displays.

Rate/Scope Tab

The Rate/Scope Tab presents two graphs, Rate Error and Amplitude, that support direct inspection of the watch signal and its derived timing measurements. The upper Rate Error graph shows the timing relationship between the tic and tac events. Ideally, the two lines should be close together and as nearly horizontal as possible. When the lines remain close, it suggests that the watch is running with relatively small beat error and stable timing behavior. If the lines separate noticeably, this may indicate increasing beat error or inconsistency in the detected beat timing. If both lines slope upward, the watch is typically running fast relative to its nominal rate; if both lines slope downward, the watch is typically running slow. Steeper slopes indicate a larger timing deviation.

The lower Amplitude graph shows the waveform features used to estimate amplitude. It identifies the onset of signal A with a dotted green line and the peak of signal C with a dotted red line. The time displayed in milliseconds above the C peak indicates the interval from the onset of A to the peak of C. A second timing measurement, shown as a double-ended arrow between green dotted lines, indicates the interval between consecutive A events. In this graph area, the user can also zoom in and out to inspect the waveform in greater or lesser detail and can move backward or forward in time to review earlier or later portions of the captured signal. Together, these displayed values, markers, and navigation features provide an initial implementation of event timing and

amplitude-related measurement, and student teams are expected to refine and improve their accuracy and robustness.

Sound Print Tab

The Sound Print Tab provides a sample-based view of the detected watch signal and highlights the key acoustic events used for measurement. In this display, the software attempts to detect the strongest relevant events in each beat sequence, beginning with A and followed by C. The detected A event is marked with green dots, and the detected C event is marked with blue dots, allowing the user to see how the event detector is tracking the waveform over time and how those detected events align with the acoustic signal. These colored markers also help the user inspect how consistently the signal is being tracked across repeated beats. If the aggregate green-dot pattern continues to rise, the display indicates that the watch is running fast; if it trends downward, the watch is running slow.

The Sound Print display is organized vertically by sample position rather than by a conventional time-axis graph. The total vertical range depends on the selected sample rate. At 48,000 samples per second, the display represents approximately 8,000 samples, while at 192,000 samples per second, it represents approximately 32,000 samples. This gives the user a denser and more detailed view of the signal at higher sample rates.

To compute, a sample rate of 192,000 samples per second, one beat interval corresponds to:

$192000 \div 6 = 32000$ samples per beat

Thus, in this case, the full vertical Sound Print window corresponds approximately to one beat interval, or 32,000 samples. At lower sample rates, the same beat interval would occupy fewer samples and therefore less vertical resolution.

The purpose of the Sound Print Tab is to help users inspect the raw or filtered signal at the sample level, verify that the A and C events are being detected at the correct locations, and better understand how sample rate affects timing precision and event resolution. Student teams should consider how this display can be improved to make event locations clearer, support interpretation at multiple sample rates, and provide more informative views of the signal structure.

Control Panel - Run Parameters

Gain
Adjusts the input signal level used for analysis. This control helps match the microphone signal to a usable range so that beat events can be detected clearly without excessive clipping or loss of detail.

Live Mode
Captures and analyzes signal data directly from the microphone in real time. This mode is used when measuring an actual watch on the timegrapher hardware.

Playback Mode
Uses a previously recorded signal instead of live microphone input. This mode is useful for reviewing past captures, debugging the software, and comparing algorithm behavior on the same data.

Sim Mode
Generates a synthetic watch signal for testing and development. This mode is useful when hardware is unavailable or when teams want controlled inputs for evaluating detection, graphing, and GUI behavior.

Refresh
Restores the run settings to their initial values. This control can be used to reset the current configuration and return the Run Parameters section to its default starting state.

Sample Rate
Sets the sampling rate in Hz used to acquire or process the signal. Higher sample rates provide more timing resolution and can improve event detection accuracy, but they also increase processing cost and memory usage.

Averaging Period
Defines the time window over which measurements are averaged before being displayed. A longer averaging period produces steadier readings, while a shorter averaging period makes the display respond more quickly to changes.

Start
Begins signal acquisition and analysis using the currently selected settings.

Stop
Stops acquisition and analysis while preserving the current display state for review.

Save
Stores the current recording, measurements, or display data for later review, debugging, or comparison.

Control Panel - Watch Parameters

BPH (Beats Per Hour)
Selects the nominal beat rate of the watch movement. This value is used in timing calculations and affects measurements such as rate, beat error, and amplitude. The drop-down menu should allow the user to choose from common beat rates or use an automatic-detection option when available.

Lift Angle
Sets the lift angle used in the amplitude calculation. Because lift angle depends on the watch movement, the GUI should provide a drop-down menu or adjustable input so the user can select or

modify the correct value. Accurate lift-angle selection is important because an incorrect setting will produce incorrect amplitude estimates.

Control Panel - Simulation Parameters

BPH (Beats Per Hour)
Sets the nominal beat rate of the simulated watch signal. This determines how frequently simulated beat events occur and should match the type of movement being modeled.

Error Rate
Sets the simulated rate deviation from ideal timekeeping. This allows the generated signal to represent a watch that is running fast or slow and can be used to test whether the GUI and calculations correctly reflect that deviation.

Amplitude
Sets the simulated balance amplitude for the generated signal. This lets teams test how the system displays and analyzes signals corresponding to stronger or weaker watch motion.

Beat Error
Sets the simulated asymmetry between the tic and tock intervals. This parameter is useful for testing whether the software correctly detects and displays beat error conditions.

Realistic
Enables a more realistic simulation of the watch signal by introducing behavior that more closely resembles an actual mechanical watch, such as variability, noise, or non-ideal signal characteristics. This mode is useful for testing how robust the detection and graphing algorithms are under more realistic conditions.

Control Panel - Miscellaneous Parameters

Low Pass
Sets the low-pass filter used to remove unwanted high-frequency noise from the signal. This can help smooth the waveform and make important beat features easier to detect.

High Pass
Sets the high-pass filter used to remove unwanted low-frequency components, such as background hum, handling vibration, or slow signal drift. This can improve the clarity of the watch signal by emphasizing the sharper beat events.

C Event Use Onset Amplitude
Sets the amplitude threshold or sensitivity used to identify the onset of the **C** event. Adjusting this value affects where the software marks the beginning or significant point of the C signal, which can influence amplitude-related measurements.

Expected Enhancements

Student teams should extend the baseline GUI to support richer interaction, additional measurements, improved filtering of the raw acoustic signal, and expanded real-time visualization capabilities.

The user should be able to pause the live display in any graph or tab and use a cursor to move backward and forward through captured readings so that individual beats, markers, and computed values can be examined in detail. This capability should retain the recorded live data as the user interacts with the graph.

Beyond these baseline enhancements, student teams are expected to implement additional real-time graphs and diagnostic displays described in the later sections of this document. These include, but are not limited to:

- Watch-Position Testing
- Trace Display
- Rate and Amplitude Stability Over Time
- Multi-Position Sequence Display
- Beat-Noise Scope Display
- Beat Error Display and Diagnostic Trace
- Long-Term Performance Graph
- Escapement Analyzer and Marker-Line Display
- Time-Frequency Spectrogram Display
- Waveform Comparison Display with Timing Markers
- Scope Mode with Synchronized Sweep Display
- Scope Function with Multiple Filter Views

The enhanced system should also compute and display additional derived timing measures as needed. Drawing inspiration from the Watch Time Parameters Measurement Screen in the Chour reference, these may include values such as DiffTicTac, the difference in duration between a tick and a tock; DiffPeriod, the average difference between the measured beat duration and the expected beat duration over a short fixed interval such as four seconds; and Avg Period, the average difference between the measured and expected beat durations accumulated from the start of measurement or since the last reset. These measures can help users see not only the overall rate of the watch, but also short-term asymmetry, short-term drift, and longer-term accumulated timing behavior. Student teams are encouraged to consider which additional derived measures would be most useful to display in the GUI and how to present them in a way that is understandable and meaningful to the user.

Student teams should enhance the GUI so that all graphs can run continuously within the same application, without requiring the user to stop and restart the program in order to view a different graph or analysis mode. The interface should support common interactive controls such as start, stop, pause, and the ability to move backward and forward in time through captured data. Users

should be able to pause the display and inspect prior data without losing the recorded signal or forcing a reset of the measurement session. The enhanced GUI should also support interactive selection of timing points and measurement regions so that users can examine intervals, compare events, and make detailed timing measurements directly from the graphs. The goal is to make the system more useful as both a real-time monitoring tool and an interactive diagnostic tool.

In short, teams should make the signal observable, measurable, and interpretable through all of these complementary views.

Sound Print Enhancement

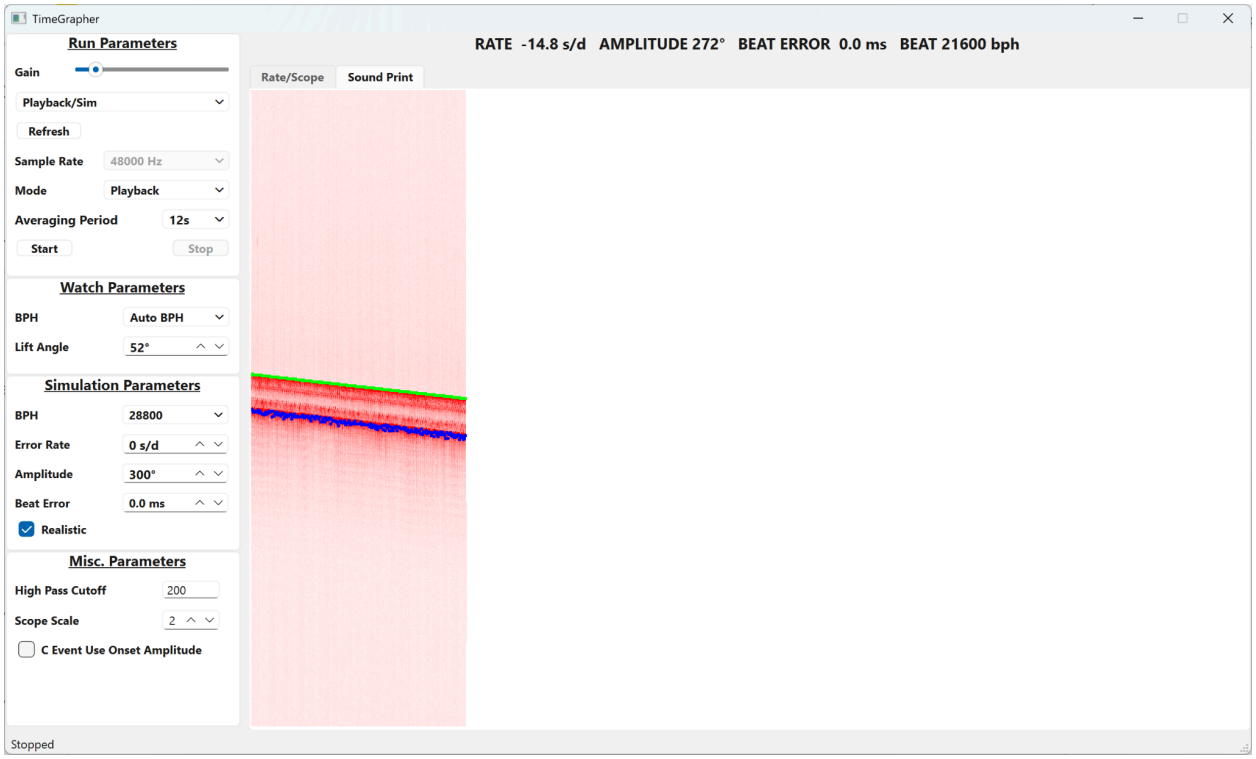

Figure 6: Sound Graph

In addition to showing the raw or processed watch signal, the graph should support clearer visualization of small timing variations, optional averaging over a user-selectable period, and improved filtering to reduce ambient noise while preserving the watch sounds needed for analysis. The display should help the user see fluctuations that may be too small to notice in a basic trace, and it should support a scope-like view of the acoustic waveform for more detailed inspection of watch faults. The graph should also make the measurement process easier to interpret by indicating the active averaging window, showing thresholds or reference lines when helpful, and supporting pause-and-review behavior so prior traces can be examined across positions or conditions. Overall, the enhanced sound graph should serve not only as a live signal display, but as a diagnostic view that helps the user understand stability, noise, and possible watch problems more clearly.

Rate/Scope Enhancement

The Rate graph is currently a rectified image of the watch signal. Student teams should consider extending this graph to display the raw watch sound waveform as well, so that users can compare the original signal with the analyzed timing view. The same raw signal should also be available in the Sound Print Tab, providing a consistent way to inspect the actual watch sound across multiple displays.

AI Feature

Student teams shall explore the use of on-device intelligence to improve signal quality, event detection, and user guidance without depending on cloud services. By running lightweight AI or TinyML models directly on the Raspberry Pi, the system could help determine whether incoming data is usable, noisy, incomplete, or misleading in real time. For example, the AI could identify bad data caused by speech, handling noise, poor microphone contact, clipping, dropped samples, or weak watch signals, and either reject those segments or flag them for the user. This could improve measurement quality by preventing unreliable data from influencing computed values such as rate, beat error, and amplitude.

Multiple use cases may be found, including but not limited to:

Signal Quality Classification
Determine whether the current signal is good, noisy, clipped, too weak, or corrupted by outside interference.

Bad Data Rejection
Detect and ignore portions of the signal that should not be used for measurement.

Fast/Slow Watch Classification
Learn from recent beat patterns and help classify whether the watch is trending fast, slow, unstable, or out of beat.

User Guidance
Provide real-time hints such as “signal too noisy,” “reposition watch,” “microphone gain too high,” or “measurement confidence low.”

Features to Build & Implement

Test Positions

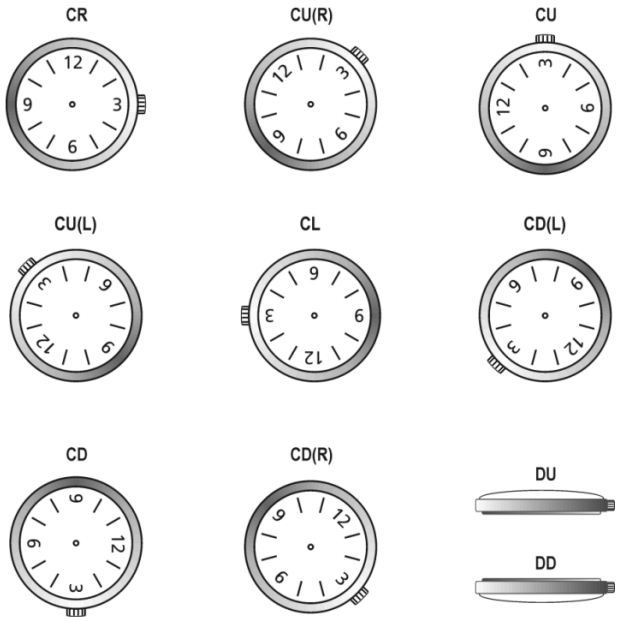

Figure 7[2]: Indication of the test positions in accordance with NIHS 95-10/ISO 3158

The system shall support testing a mechanical watch in all standard measurement positions and shall identify and display the current position in the GUI. The position set should follow the watch-position conventions shown in the Chronoscope X1 (G3) manual (reference c) , including the horizontal positions CH and CB and the vertical positions 6H, 9H, 3H, and 12H, with support for intermediate positions when used. The GUI shall clearly indicate the active test position while measurements are being taken so that the user always knows the orientation associated with the displayed results.

Graphs to Improve or Create

Student teams are expected to reproduce and adapt many of the graphing and display concepts shown in the Witschi Chronoscope X1 G3 Instruction Manual (reference c). These graphs and displays should serve as functional inspiration for the project rather than as exact copies. The goal is to implement comparable real-time visualizations that help users interpret watch behavior, timing performance, stability, and signal characteristics. All graphs shall update in real time as measurements are acquired and processed

[2] Page 13, Witschi Chronoscope X1 G3 Instruction Manual

Trace Display

Figure 8[3]: Trace Display

Student teams shall implement a trace display, inspired by the Trace Display Mode and related long-term stability views in the Witschi Chronoscope X1 G3 Instruction Manual (reference c). The trace display shall continuously record and display rate deviation and amplitude over time in real time. The GUI may present these as two stacked graphs or as two separate graphs, provided both measurements are clearly visible and easy to interpret.

To improve readability, the trace display shall include a smoothing function for the **s/d** reading so that short-term fluctuations do not make the graph difficult to interpret. The system shall alert the user when the rate indicates that the watch is running late. The amplitude portion of the display shall show whether the watch remains within a normal operating range, which for this project should generally be treated as 270° to 300°, and shall alert the user when amplitude falls outside that range. The GUI shall also include short explanatory text or labels to help users understand how to read the graph outputs.

In addition to the real-time display, the system shall support longer-term summaries for both measures, including an average over an extended period such as a day and a rolling average that updates over time. The purpose of this display is to help the user evaluate both short-term behavior and longer-term stability of the watch from the same interface.

[3] Page 14, Witschi Chronoscope X1 G3 Instruction Manual

Rate and Amplitude Stability Over Time

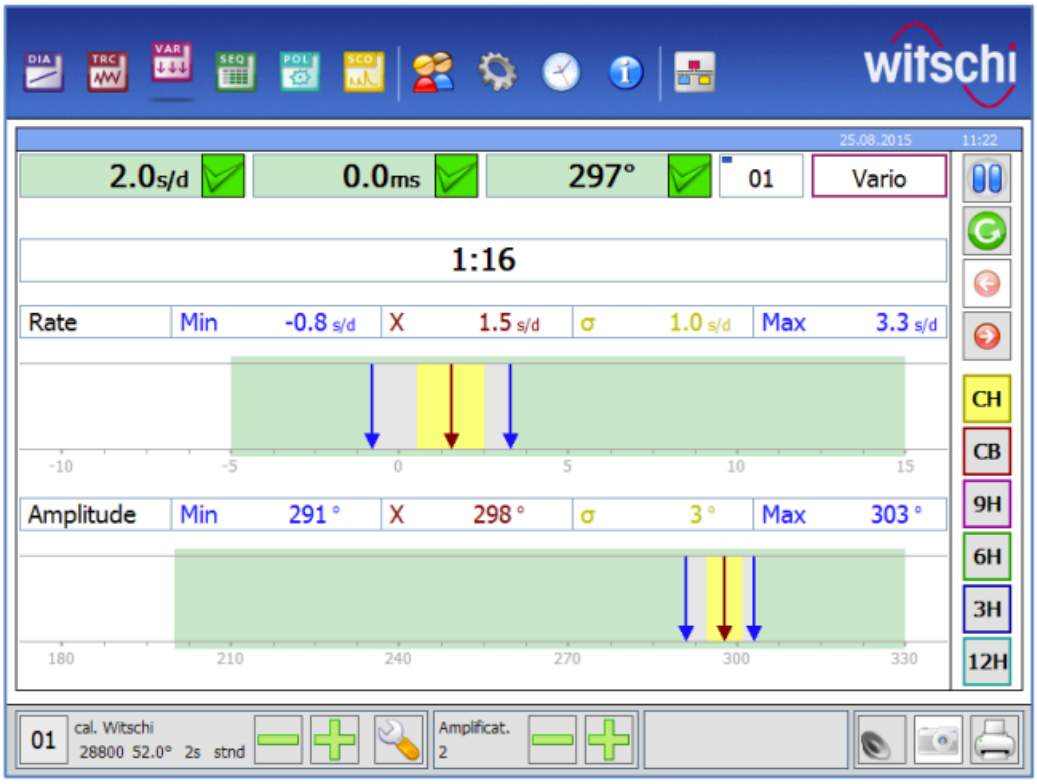

Figure 9[4]: Vario Display

Student teams shall implement a Vario Display that shows the long-term stability of both rate and amplitude over time, based on the example in the Witschi Chronoscope X1 G3 Instruction Manual (reference c). This display shall continuously update key statistical values during measurement, including the minimum, maximum, average, standard deviation, elapsed measurement time, and current reading for both rate and amplitude. The purpose of this display is to help the user evaluate watch quality and consistency over a longer period rather than only viewing instantaneous values. In particular, a smaller difference between the minimum and maximum rate values indicates better stability, the average value helps assess the overall adjustment quality of the watch, and sigma indicates how much the measurements vary over time. The GUI should clearly distinguish acceptable ranges and measured values, for example by using a green region to indicate acceptable min/max ranges, blue arrows to mark the measured minimum and maximum values, and a red arrow to identify the average value.

[4] Page 15, Witschi Chronoscope X1 G3 Instruction Manual

Multi-Position Sequence Display

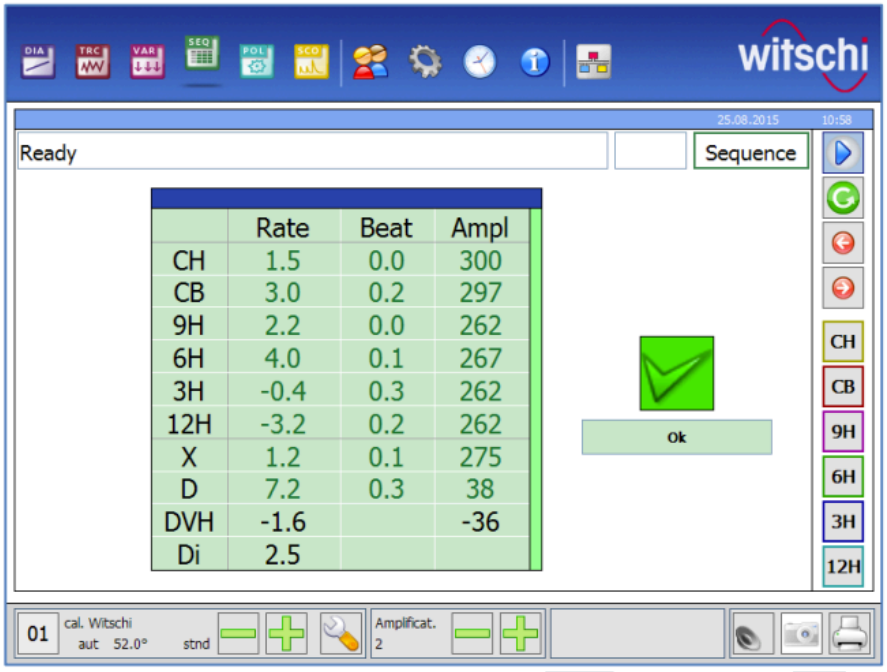

Figure 10[5]: Sequence Display Mode

Student teams shall implement a Sequence Display Mode that supports a complete measurement cycle across multiple watch test positions, based on the example in the Witschi Chronoscope X1 G3 Instruction Manual (reference c). The display shall allow results from up to 10 test positions to be captured and reviewed in a single sequence. For each position, the system shall calculate and display measurement results for rate, amplitude, and beat error, and it shall also compute summary values across the sequence. At a minimum, the display shall show X, the mean of all test positions, and D, the difference between the largest and smallest measured value. The system should also support comparisons between vertical and horizontal positions and include indicators that help reveal possible balance-wheel unbalance when applicable. The purpose of this display is to help the user compare watch behavior across positions and assess positional stability and consistency. Features or codes intended only for special watch types are not required for this project.

[5] Page 15, Witschi Chronoscope X1 G3 Instruction Manual

Beat-Noise Scope Display

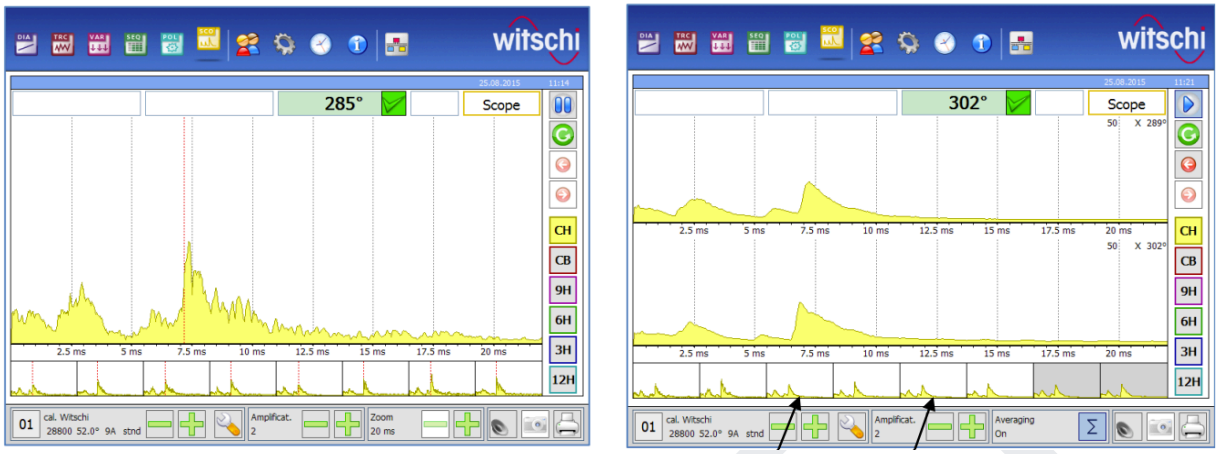

Figure 11[6]: Scope Display Mode, Scope 1 & Scope 2

Student teams shall implement a Scope Display Mode with two related views based on the examples in the Witschi Chronoscope X1 G3 Instruction Manual (reference c). In the figure, Scope 1 is shown on the left and Scope 2 is shown on the right. Together, these two views are intended to help the user inspect the detailed shape, timing, and repeatability of the watch's acoustic beat noises rather than relying only on summary measurements.

In Scope 1 (left figure), the GUI shall graphically display the watch's alternating tick and tock beat noises for detailed inspection. The display shall support selectable time ranges of 20 ms, 200 ms, and 400 ms. After sufficient measurement time, the most recent beat noises shall appear as small strips beneath the current waveform, and the user should be able to select one of these prior beats for enlarged viewing. The signal may also be displayed as its absolute value when that improves readability. The display shall identify the relevant A and C beats, including a visual marker for the C beat as shown in the reference figure, and shall present the lift angle associated with the displayed beat pattern.

In Scope 2 (right side of the figure), the GUI shall display tic and tac beat noises on two horizontal axes using a fixed 20 ms time range and shall allow the user to turn averaging on or off with a Σ control. When averaging is enabled, multiple beat noises shall be combined to reduce random noise and improve signal clarity. The duration of the measurement cycle shall depend on the watch's beat number and the selected interval, and the cycle shall complete after 50 tic and 50 tac intervals. At the end of the cycle, the system shall display the average amplitude on each horizontal axis. In the reference figure, the arrows indicate these average amplitude values for the two displayed traces. Because the system does not guarantee which axis corresponds to tic and which corresponds to tac, the GUI should present them as the two averaged beat-noise traces rather than assuming a fixed assignment. The interface may also show intermediate averaging results, such as averages after 10 intervals or 20 intervals, as illustrated in the reference.

[6] Page 19, Witschi Chronoscope X1 G3 Instruction Manual

Beat Error Display and Diagnostic Trace

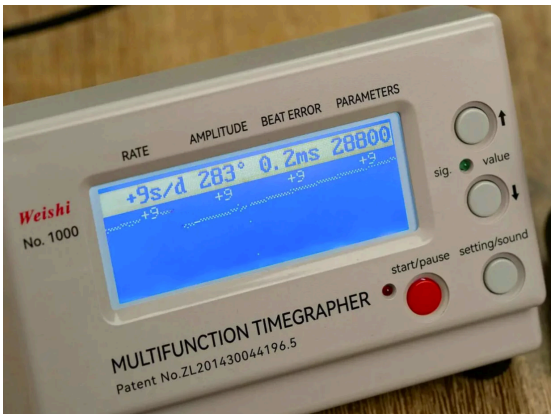

Figure 12: A Standalone Weishi No 1000 Display

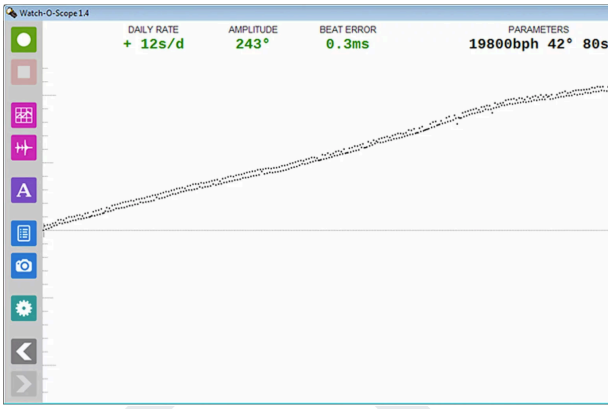

Figure 13: Watch-O-Scope GUI Showing Beat Error

Student teams shall implement a Beat Error Display and Diagnostic Trace that presents both numerical measurements and a corresponding graphical trace for diagnostic interpretation. Inspired by the standalone timegrapher display and the Watch-O-Scope graphing approach, the GUI shall show numeric values for rate, amplitude, beat error, and beats per hour, together with one or more trace lines that visually represent the timing behavior of the watch. The line display shall be consistent with the numeric values; for example, a positive reading shall correspond to a positively sloped trace. The desired condition is for the displayed line or lines to remain as close to horizontal as possible. When two lines are shown, the system shall alert the user if their separation exceeds an acceptable range. If the slope becomes excessively positive or negative, such as greater than 45 degrees in magnitude, the GUI shall indicate a major fault condition. This feature is intended to help users identify beat error, instability, and likely adjustment problems more easily than with numeric readings alone.

Long-Term Performance Graph

Figure 14: Performance Graph Over Time

Student teams shall implement a Long-Term Performance Graph that records and displays how a watch's rate, amplitude, and beat error change over an extended period of time. The purpose of this graph is to help the user observe fluctuations that may not be visible during a short measurement, including changes caused by power reserve, complications such as date change, or other cyclic behaviors. The graph should update periodically during the test, show the overall average for the testing period, and visually indicate the range of typical variation so that the user can quickly judge how consistent the watch is over time. The display should also support longer test durations by reducing update frequency as elapsed time increases, allowing the system to monitor performance over many hours while remaining readable and efficient.

Escapement Analyzer and Marker-Line Display

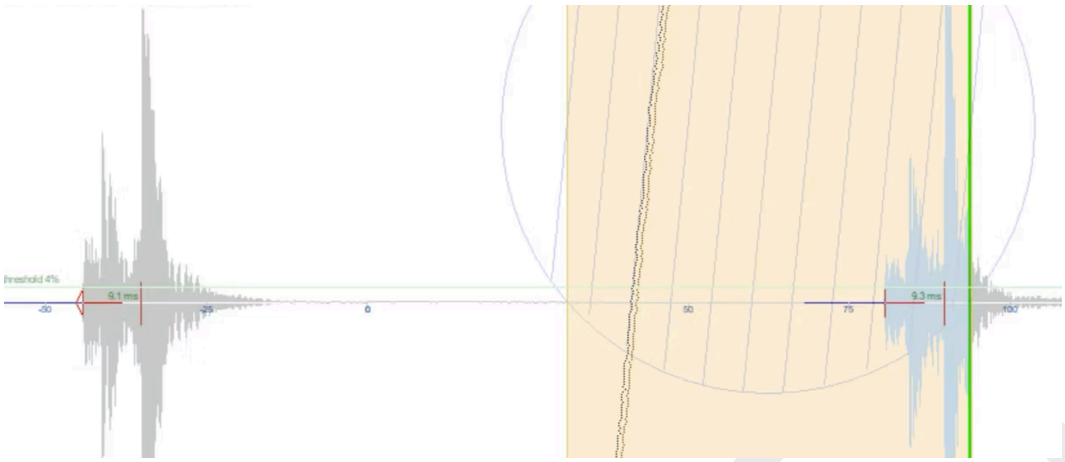

Figure 15[7]: Watch Escapement Analyzer

Student teams shall implement an Escapement Analyzer and Marker-Line Display that allows the user to inspect the detailed timing relationships within each watch beat. Inspired by the reference figure, this display shall present the acoustic waveform together with vertical timing markers and millisecond labels that identify important events within the escapement cycle. At a minimum, the system shall support placing and displaying markers for the relevant A and C events and shall calculate the elapsed time between them in milliseconds. The purpose of this display is to help the user examine fine-grained beat timing that is not visible in higher-level summary graphs.

The GUI shall make the marker lines easy to view and interpret in real time. Marker positions should update consistently with the waveform and remain visually aligned with the signal features they represent. The interface should allow the user to compare alternative interpretations of the signal, such as whether timing is more repeatable when measured from the start/onset of a feature or from its peak. This is important because one goal of the display is to help determine which reference point produces the most stable and meaningful timing results.

[7] https://www.etimer.net/#adi_page117y_1_115

Time-Frequency Spectrogram Display

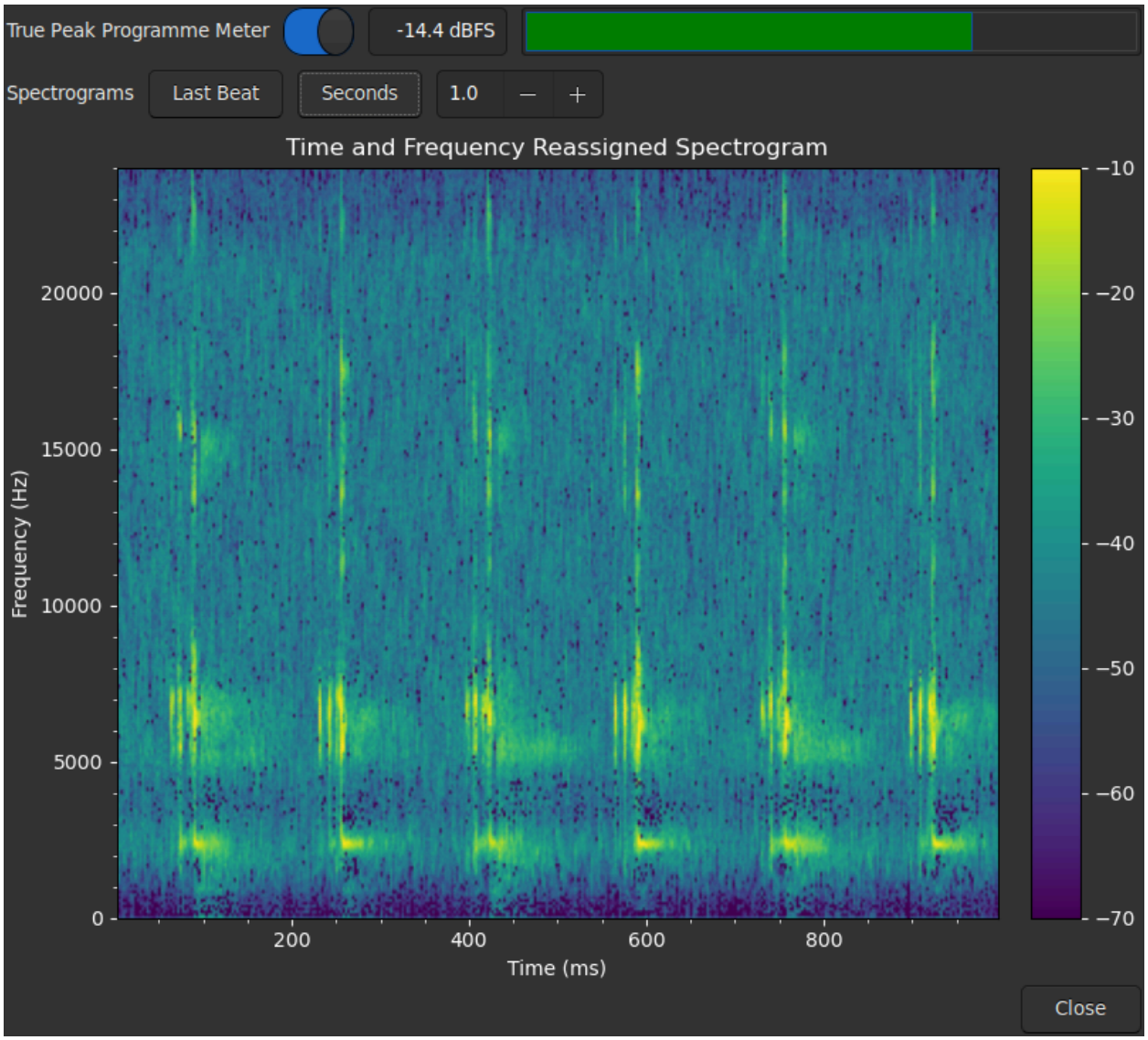

Figure 16: Time and Frequency Reassigned Spectogram

Student teams shall implement a Time-Frequency Spectrogram Display that shows how the watch's acoustic energy is distributed across both time and frequency. Inspired by the reference figure, this display shall present a spectrogram in which the horizontal axis represents time, the vertical axis represents frequency, and color intensity represents signal strength. The purpose of this display is to help the user identify repeating beat patterns, distinguish important acoustic components, and observe how different frequency bands behave during each tick and tock event.

The spectrogram should support real-time updating and allow the user to inspect either the most recent beat or a selected recent time window. The display should make it possible to see recurring structures in the watch sound, such as repeated bursts of energy at characteristic frequency ranges,

and to compare one beat with the next. The GUI should also include a color scale or legend so that users can interpret relative signal intensity across the graph.

Waveform Comparison Display with Timing Markers

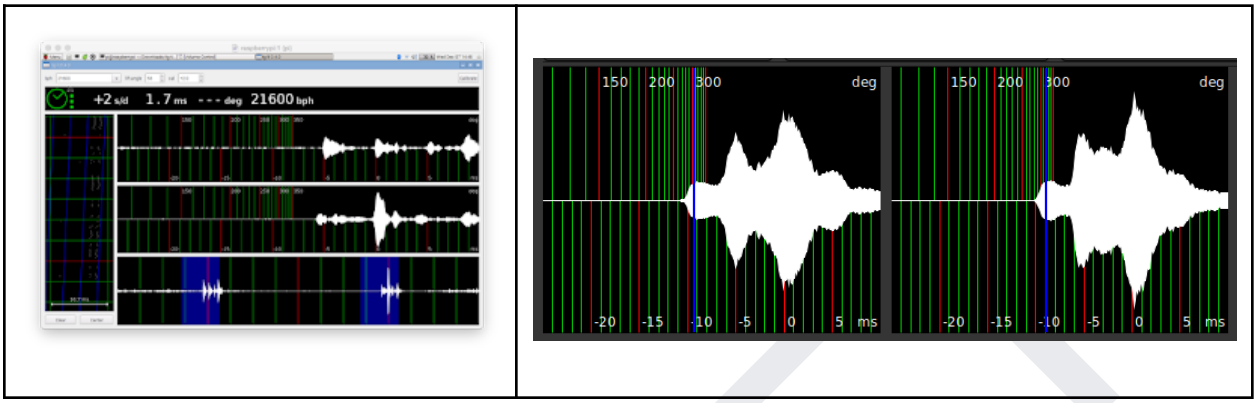

Figure 17[8]: Waveform Images from Marcello Mamino's Tg timegrapher

Student teams shall implement a Waveform Comparison Display with Timing Markers that presents multiple beat waveforms in aligned lanes so that users can compare their shape, spacing, and consistency. As shown in the reference images, the GUI should overlay the waveform with vertical guide markers and display key numeric values such as rate, beat error, and beats per hour. The display should allow users to compare successive beats, identify important landmarks in the signal, and inspect how waveform structure changes from one beat to the next. The system should also help users decompose the signal in this way, making it easier to see the signal envelopes and understand what is happening within each beat. Where appropriate, the interface may also include degree-based or time-based reference markers to help interpret amplitude and timing relationships. The purpose of this feature is to provide a more informative diagnostic view than a single raw waveform by combining repeated beat comparison, timing landmarks, and measurement context in one display.

[8] https://tg.ciovil.li/

Scope Mode with Synchronized Sweep Display

Figure 18: One Second Synchronization

Student teams shall implement a Scope Mode that displays the watch's acoustic signal in real time in a fixed sweep window, similar to an oscilloscope. The display should use the processed signal that combines the upper and lower halves of the waveform so that beat features are easier to see. The sweep time should be configurable as a multiple of the watch's tick interval so that repeated beats appear in stable positions on the screen. When the watch is running close to its nominal rate, the beat pattern should remain visually stable; if the watch is fast or slow, the pattern should drift across the display. The GUI may also show reference values such as daily rate, amplitude, beat error, and nominal beat rate from the most recent timing test so the user can compare the live waveform against the most recent measurements.

Scope Function with Multiple Filter Views

Figure 19: PC-RM4 Four Scope Filters

Student teams shall implement a Scope Function to support four filter views, labeled F0, F1, F2, and F3, based on the approach from https://www.pascalchour.fr/mesures/chour_rm4_en.htm. These views shall allow the user to examine the same watch signal under different forms of processing so that both the raw waveform and important beat features can be inspected more effectively. The GUI should make it easy to switch among the four filters and compare how each one changes the appearance of the waveform and the visibility of key events such as T1, T2, and T3.

F0 shall display the signal as captured, formatted to fit the screen and mirrored around its average value so that both positive and negative excursions are reflected symmetrically. Although the sensor and amplifier may already influence the waveform through their own filtering, this view should be treated as the closest representation of the watch signal available to the system.

F1 shall apply a moving-average filter to the F0 signal. This view should smooth the waveform envelope, reduce a large portion of the background noise, and provide a more visually readable trace. However, the system documentation should note that low-amplitude signal components may become less visible in this mode.

F2 shall build on F1 and apply additional processing that emphasizes rising slopes while attenuating falling slopes. This filter is intended to make important beat features stand out more clearly, especially T3, and to some extent T2. The implementation may use an attenuation function that decays after a local rise so that sharp upward features remain prominent while the following decay is suppressed.

F3 shall display only the upper portion of the signal relative to its average value, bringing the lower portion upward and applying emphasis to rising edges together with attenuation on their falling portions. Although this view may be less visually intuitive than the others, it can be useful for identifying T1 and especially T3.

Together, these filter views should help the user inspect the waveform from multiple perspectives: F0 for the closest raw representation, F1 for smoothing and readability, F2 for emphasizing major beat landmarks, and F3 for feature detection. The purpose of this feature is to improve signal interpretation, reduce the impact of background noise, and support more reliable identification of important timing events.

Optional - Advanced Setup and Configuration

The provided GUI already includes several configuration controls, such as drop-down menus and text-entry fields, and student teams should use their imagination to extend these capabilities in ways that improve usability and diagnostic power. The eTimer (reference G) setup provides useful inspiration for advanced configuration features, including customizable sampling parameters, sound-device and port selection, long-term data logging, optional environmental logging using Arduino-based temperature, pressure, and humidity sensors, and calibration support using either an internet time server or a GPS dongle for improved accuracy. Student teams are not expected to

copy this interface exactly, but they should consider how similar setup and configuration options could make the TimeGrapher more flexible, informative, and easier to use.

Quality Attributes

Real Time Performance

The system shall acquire, process, analyze, and display watch acoustic data in real time on the Raspberry Pi while maintaining a responsive GUI. The architecture should support sustained processing at multiple sample-rate targets, with 96,000 samples per second as the objective, 48,000 samples per second as the minimum acceptable threshold, and 192,000 samples per second as a stretch goal. Because the system must run on Raspberry Pi hardware, teams should manage memory carefully and create designs that avoid running out of memory or bogging down the machine that does not have enough memory.

Low Latency and Low Number of Missed Beats

The system shall minimize end-to-end latency between acoustic capture at the microphone and presentation of the corresponding waveform, markers, and computed values in the GUI. The system shall record the time difference between (1) when an audio sample block is captured, (2) when that block is processed for beat detection and measurement, and (3) when the corresponding waveform segment and computed readings are displayed in the GUI. The difference between capture time and display time is the end-to-end latency. Teams shall report capture-to-processing latency, processing-to-display latency, and total end-to-end latency in milliseconds, together with average and worst-case values, as well as counts of dropped audio blocks and missed beat detections, so that stale data, backlog, and timing failures can be observed directly.

Correctness

The system shall compute watch-performance measures correctly and consistently from the captured acoustic signal, and the displayed values and graphs shall correspond to the underlying watch events while remaining internally consistent across the GUI, derived measurements, and longer-term summaries. Because the project includes rate- and amplitude-stability graphs, beat-error views, sequence displays, and marker-based analysis, the architecture should make it possible to verify that calculations and visualizations are based on the same underlying data and timing assumptions. The system shall also remain usable in the presence of ambient acoustic noise and other signal disturbances by using filtering and related preprocessing techniques that reduce extraneous noise, such as nearby speech, while preserving the features needed for correct beat detection and measurement.

Measurement Accuracy, Error Detection, and Handling

The system shall detect the relevant watch events with sufficient accuracy to support meaningful measurement of small timing differences. In particular, the software must accurately identify the start/onset and peak of the important acoustic signals used to compute watch metrics such as rate, beat error, amplitude, lift angle, beats per hour, and balance-wheel frequency. Since the project depends on deriving measurements from specific watch sounds, the architecture should preserve timing precision throughout acquisition, filtering, event detection, and calculation. Since these measurements are derived from very small timing differences at high sample rates, slight deviations may be significant. Accordingly, the system should degrade gracefully when the signal is weak, noisy, or partially missing, rather than producing unstable or misleading outputs.

Extensibility, Modifiability

The system shall be easy for student teams to understand, extend, test, and debug within the limited project schedule. Its architecture shall support the addition of new measurements, filters, graphs, and display modes without major redesign of existing code. Because teams begin with a baseline GUI and are expected to add substantial new capabilities, the design should separate signal acquisition, signal processing, calculation, presentation, and platform-specific concerns so that enhancements can be implemented incrementally, tested independently, and added with limited impact on existing modules on both the PC and Raspberry Pi platforms.

System Hardware

Raspberry Pi

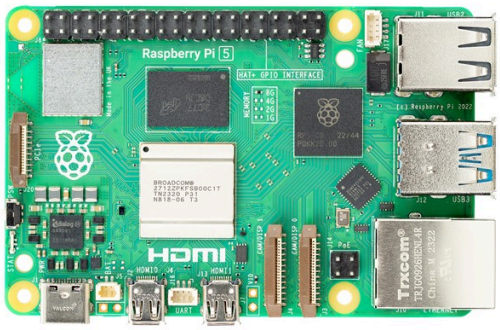

Figure 20: Raspberry Pi

The CanaKit Raspberry Pi 5 Starter Kit is a beginner-friendly package built around the Raspberry Pi 5, a compact single-board computer designed for learning and hands-on computing projects.

The Raspberry Pi 5 functions like a small desktop computer, capable of running Linux-based operating systems and supporting tasks such as programming, web browsing, data processing, and hardware interfacing. In this configuration, it includes **8 GB of RAM**, allowing for smoother multitasking and improved performance in more advanced applications, such as real-time data processing and lightweight development workloads.

Storage is provided through a **128 GB microSD card**, which holds the operating system, software, and user files, functioning similarly to a solid-state drive in a traditional computer.

8 Inch Touchscreen for Raspberry Pi

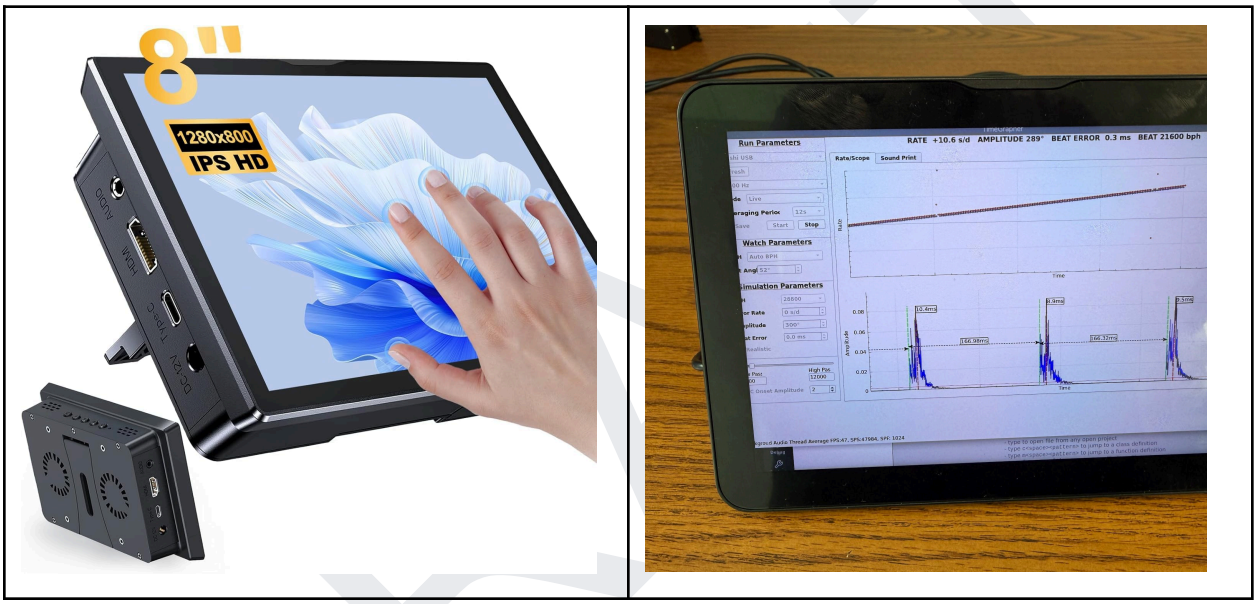

Figure 21: Touchscreen for Raspberry Pi

The Raspberry Pi 5 touchscreen is a compact 5-inch capacitive display with 800×480 resolution, designed for use with Raspberry Pi systems. It connects via HDMI (video) and USB (touch input), allowing it to function as an interactive monitor. It supports plug-and-play setup for most Raspberry Pi projects and is commonly used for dashboards, control panels, and embedded computing applications where a small, responsive interface is needed.

Time Grapher

Figure 22: Microphone of Weishi Timing Timegrapher

The microphone section of a Weishi-style timing timegrapher is the mechanical-acoustic transducer assembly that couples the watch to the measurement system. Its function is to detect the small impulsive vibrations generated by the escapement and convert them into an electrical signal with sufficient amplitude, bandwidth, and signal-to-noise ratio for downstream processing.

The assembly includes a watch support fixture, a contact or vibration-sensitive pickup, and a mechanical housing that helps maintain repeatable positioning of the watch relative to the sensor. The fixture constrains the watch securely while avoiding excessive damping or added vibration. The pickup is optimized for impulsive, low-level mechanical events rather than airborne sound alone, since direct mechanical coupling improves rejection of ambient acoustic noise. The microphone output is then passed to an analog front end that may include gain, filtering, and impedance matching before digitization.

In the context of a timegrapher, this subsystem is critical because errors introduced at the microphone stage, such as poor coupling, excessive gain control, vibration contamination, or

filtering that distorts transient features, can directly degrade the accuracy of detected beat timestamps and therefore affect computed values such as rate, beat error, and amplitude.

System Software

GUI Code

All files and source code needed to build, run, and modify the GUI in Qt will be provided in a zip file named TimeGrapher_v10.4_Student.zip. This archive will contain the existing TimeGrapher GUI implementation and the supporting project files needed for student teams to open the project in Qt Creator, build it, and begin making enhancements.

Qt and Qt creator

Student teams will use **Qt Creator** to design and implement improvements to the TimeGrapher graphical user interface. Qt Creator is the integrated development environment commonly used for building **Qt-based desktop applications**, including user interfaces, widgets, layouts, and application logic. For this project, students should use it to extend the existing GUI, add controls and displays, and improve usability while keeping the software organized and maintainable.

The open-source Qt installer is available from Qt for several operating systems and requires a Qt account for installation.
https://www.qt.io/development/download-qt-installer-oss

Raspberry Pi OS

The Raspberry Pi will be provided with a standard Raspberry Pi 5 system image suitable for project development and testing. The image will include the necessary base operating system and development environment needed to run the project on the target hardware. The existing TimeGrapher GUI code will already be installed on the Raspberry Pi so that student teams can begin by studying, running, and extending the provided implementation rather than building the system from scratch.

To ensure proper operation, student teams must verify that Auto Gain Control (AGC) is turned off. If AGC remains enabled, it can distort or suppress the microphone input and cause the TimeGrapher to perform unreliably. See Figure X below.

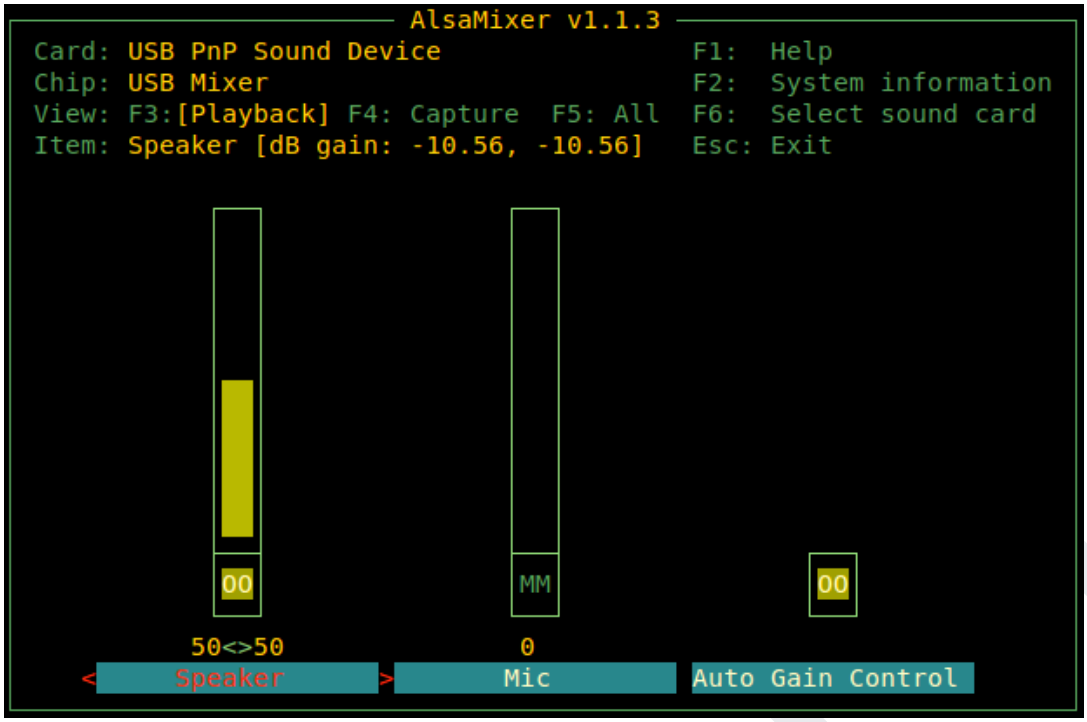

Figure 23: AlsaMixer Display to Adjust Auto Gain Control

Project Deliverables

There are three milestones required for this project (each of which will be graded independently):

1. **Requirements, project plan, plan for experimentation, and risks**: For the first milestone the team will turn in the prioritized architectural drivers, any draft design decisions they are considering, the technical risks identified, and the experimentation plans (or any results already achieved).
2. **Experimentation results, design, plan for construction**: The team will turn in the results of experiments conducted, a design description with different architecture views, and the plan for construction.
3. **Demo and lessons learned**: For the final milestone, the team will demo the completed system, and present the lessons learned.

Milestone 1

The submission can be informal documents describing the team's current understanding of the items listed below. Team mentors will be meeting with the teams to ask follow up questions.

- Project Plan

    - The plan should describe the division of roles, the specific tasks planned, and the associated milestones.
    - The plan should reflect construction tasks based on the overall architecture.
    - The tasks should also reflect planned technical experiments.
- Architectural Drivers
    - Are the quality attribute requirements “actionable”? In other words, are they expressed in such a way that the team will be able to determine if a given design supports these drivers or not?
    - Do the drivers seem to relate to the overall objectives of the project?
    - Are the measures clearly derived from the overall goals of the project?
    - Are the functional requirements understood?
    - Are requirements prioritized?
- Risk Assessment
    - What are the technical and non-technical risks? How do you assess each risk with respect to probability and impact in a High-Medium-Low scale?
    - Are the open questions/issues clearly related to things that will affect the outcome of the project?
    - Have there been any actions identified to address the open questions/issues?
- Planned Experiments
    - Are the technical experiments articulated concretely and following a template?
    - Is it clear what question/issue is being addressed by the experiments?
    - Will it be clear when the experiments are complete?
- Architectural Approaches
    - What is the overview-level description of the architecture?
    - What are the main architectural approaches (tactics, patterns, design strategies) in your solution?
    - Is the proposed design sound enough to guide construction?
    - Are the architectural approaches clearly related to the drivers (will they likely impact the properties of interest)?
    - Can you explain how well the architectural drivers are supported by the architecture?

Milestone 2

Again, the submission can be informal documents describing the team’s current understanding of the items listed below. The mentors will be meeting with the teams to ask follow up questions.

- Project Plan
    - How has the plan changed?
    - Has the team been actively assessing risk and updating the plan accordingly?
    - Does the team have a plan for any remaining significant issues/risks?
    - Does the team have a reasonable construction plan?
- Experiments/Results
    - What experiments have been conducted?
    - Have the results of the experiments addressed the open questions/issues?
    - What experiments remain?

    - Are the experiments focused on issues relevant to the overall goals of the system?
- Architecture
    - What is the architecture in terms of the organization of code units and their dependencies? (The team shall create at least one module view of the architecture.)
    - What is the architecture in terms of components and connectors (runtime perspective)? (The team shall create at least one runtime/C&C view of the architecture.)
    - What is the architecture in terms of the supporting infrastructure (deployment perspective)? (The team shall create a deployment view that shows high-level component allocation to hardware elements and communication channels.)
    - Have the experiments led to a refinement of the architecture?
    - Does the team understand the architectural approaches they selected and respective tradeoffs?
    - Do the architectural approaches align with the goals of the system?
    - Are there significant concerns that have not been addressed?
    - Has the architecture been evaluated?

Milestone 3

For the final deliverable there will be both a team presentation and a demonstration of the final system.

Team Presentation

The presentation should cover:

- Quality attribute requirements for the system. Select a few QA requirements that were ranked as high priority and most influenced the architecture.
- Architecture description showing architecture views and highlighting key architectural approaches adopted and their rationale.
- Description and results of experiments and architecture evaluation activities.
- Lessons learned (what went right, what went wrong, what would you have done differently).

The presentation duration is 20 minutes (subject to change). That is not enough time for an extensive coverage of the topics listed above. Therefore, you should select one or a few key points in each topic to talk about.

Final Demonstration

For the final demonstration, student teams shall present the working **TimeGrapher GUI** running on the target platform and show how their added features extend the provided baseline interface. Using the existing GUI as a foundation, teams should demonstrate the new graphs, displays, and controls they implemented, explain what each addition is intended to show the user, and illustrate how the interface supports interpretation of the watch signal and derived measurements in real time. The demo should make clear how the new visualizations were integrated into the existing application rather than built as a separate prototype.

In addition to showing the new GUI capabilities, teams shall demonstrate how their design addresses the key quality attributes of the project. In particular, they should present evidence of **low latency** between signal capture and display, **real-time performance** on the Raspberry Pi, **consistency** and stability of displayed measurements, **accuracy** of signal detection and computed values, and **extensibility** of the software architecture for adding further analyses and graphs. Teams should use the demonstration not only to show that the software works, but also to explain how their architectural and implementation choices support these qualities.

Grading Rubric

The grading rubric will be distributed as a separate document and is expected to be provided during **week 2 or week 3** of the project timeline.

Reference Material

a) Witschi Electronic Ltd Training Course
Students should consult the **Witschi Training Course: Measuring Technology and Troubleshooting for Watches** for background on watch measurement concepts, interpretation of displays, diagrams, and practical troubleshooting techniques.

b) Removed - Incorporated into Reference D.

c) Witschi Chronoscope X1 G3 Instruction Manual
This manual serves as a primary reference for display modes, measurement concepts, position testing, scope displays, sequence displays, and other GUI and analysis features relevant to this project.

d) TimeGrapher Equations_v0.pdf
This document provides the equations and worked examples used to compute key measurements such as rate, rate deviation, beat error, amplitude, and related timing quantities.

e) Watch-O-Scope User Manual
The Watch-O-Scope User Manual may be used as a secondary reference for GUI behavior, scope displays, long-term testing concepts, and software-based filtering.
Reference URL: https://www.watchoscope.com/manual.html

f) eTimer Website / Escapement Analyzer Reference
The eTimer website may be used as a supplemental reference for escapement-analysis concepts, marker-line displays, and related advanced timing views.
Reference URL: https://www.etimer.net/#adi_page117y_1_115

g) eTimer Setup Instructions 10-18-22.pdf
This document may be used as a reference for advanced configuration ideas, including sampling setup, logging options, auxiliary sensor integration, COM-port configuration, and calibration support.

Assumptions and Hints

The **Witschi Training Course** provides a useful overview of mechanical-watch measurements and explains how to read the kinds of displays and graphs referenced throughout this project. Students are encouraged to use it as a background reference when interpreting timing results, amplitude behavior, beat error, and other diagnostic views.

In particular, **pages 14 and 15** of the Witschi Training Course provide guidance on how to read and interpret many of the graphs presented in this project plan, while **pages 16 through 19** discuss error detection using the **scope function**. These sections are especially helpful for understanding how waveform shape, timing markers, and display patterns relate to watch condition and adjustment.

Students should also be aware that some supplementary materials on **signature analysis** are provided as additional reference material. These may help teams think more deeply about waveform comparison, feature detection, and signal interpretation, but they are not required in order to complete the baseline system.

Finally, teams should think carefully about implementation details such as **frames per second** and **samples per frame**, since these choices will affect responsiveness, graph smoothness, processing cost, and the accuracy of real-time visualization.